

Zero to Hero Study Handbook: ask-my-docs-rag

This handbook is a repository-grounded learning guide for **ask-my-docs-rag**, a local Retrieval-Augmented Generation (RAG) system that combines retrieval, answer generation, observability, and regression gating.

Module 1: Foundations & Architecture

1.1 What this project does

ask-my-docs-rag implements an end-to-end local RAG workflow with four major capabilities:

1. Ingest Markdown/TXT documents and build a hybrid retrieval index (BM25 + FAISS): **ingest** command.
2. Answer grounded questions using retrieved context and local Ollama generation: **ask** command.
3. Capture request-level observability (traces + metrics in DuckDB): **ask** and **eval** flows.
4. Evaluate quality/latency/cost and apply CI regression gates against thresholds and a baseline: **eval --gate**.

Primary use cases:

- Build a local, reproducible RAG prototype with production-style telemetry.
- Track model quality and cost over time before shipping changes.
- Enforce quality/performance/cost constraints in CI via regression gating.

1.2 Core paradigms and patterns used here

Definitions first, then where they appear in this repo:

1. Object-oriented orchestration Definition: behavior is grouped into stateful classes that coordinate a multi-step process. Where used:
 - **RAGPipeline** orchestrates retrieval, generation, metrics persistence, and quality updates.
 - **HybridRetriever**, **OllamaGenerator**, and **MetricsStore** encapsulate subsystem behavior.
2. Functional metric computation Definition: deterministic, side-effect-free functions compute values from inputs. Where used:
 - **exact_match_score**, **token_f1_score**, **retrieval_recall_at_k** in `src/ask_my_docs/evaluation/metrics.py`.
3. Data-model-first design Definition: typed records define data contracts between modules. Where used:
 - Dataclasses in `src/ask_my_docs/models.py`: **Document**, **Chunk**, **RetrievedChunk**, **RAGAnswer**.

- RequestMetricRecord for observability row schema.
4. Pipeline architecture Definition: data moves through ordered stages where each stage transforms or enriches results. Where used:
 - docs -> chunks/embeddings/index -> retrieved chunks -> prompt -> generated answer -> trace + metrics -> evaluation aggregate -> gate.
 5. Configuration via typed settings Definition: runtime behavior is controlled by structured config loaded from environment variables. Where used:
 - AppSettings in src/ask_my_docs/settings.py with ASK_MY_DOCS_ prefix.
 6. Local-first observability Definition: traces and metrics are persisted locally for auditability and offline analysis. Where used:
 - JSONL traces via JsonLineSpanExporter.
 - DuckDB metrics via MetricsStore.

1.3 Architecture and component interaction

Main components:

- CLI Layer: src/ask_my_docs/cli.py (ingest, ask, metrics-summary, eval).
- Retrieval Layer: src/ask_my_docs/retrieval/hybrid.py (HybridRetriever, load_documents).
- Generation Layer: src/ask_my_docs/llm/ollama.py (OllamaGenerator).
- Pipeline Layer: src/ask_my_docs/pipeline.py (RAGPipeline).
- Observability Layer:
 - src/ask_my_docs/observability/tracing.py (JSONL traces).
 - src/ask_my_docs/observability/metrics_store.py (DuckDB metrics).
 - src/ask_my_docs/observability/cost.py (token cost estimation).
- Evaluation/Gating Layer:
 - src/ask_my_docs/evaluation/runner.py.
 - src/ask_my_docs/evaluation/metrics.py.
 - src/ask_my_docs/evaluation/gating.py.

ASCII architecture (main runtime flow):

```
data/docs/*.md|*.txt
-> load_documents(docs_dir)
-> HybridRetriever.from_documents(...)
-> save(index_dir): chunks.jsonl + embeddings.npy + semantic.index + manifest.json

question
-> RAGPipeline.answer(question, top_k)
```

```

-> HybridRetriever.search(query, top_k)
-> OllamaGenerator.generate(question, retrieved)
-> citation extraction + cost estimation
-> MetricsStore.record(RequestMetricRecord)
-> traces.jsonl export via OpenTelemetry
-> RAGAnswer / optional JSON payload

data/eval/eval_set.jsonl
-> load_eval_examples(path)
-> run_evaluation(pipeline, examples, top_k)
    -> pipeline.answer(...) per sample
    -> token_f1_score + exact_match_score + retrieval_recall_at_k
    -> pipeline.update_quality_metrics(...)
-> save_eval_report(output)
-> evaluate_gate(current, baseline, thresholds) [optional]

```

Module 2: Repository Map

Focus files for new contributors (first-priority learning surface):

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
pyproject.toml	Packaging, dependencies, CLI entrypoint, lint/type/test config	[project.scripts]ask-my-docs = "ask_my_docs.cli:app"	Python >=3.11, dependencies, optional dev extras
README.md	User-facing architecture and command usage	CLI command sequences and expected outputs	Env var docs (ASK_MY_DOCS_*) and workflow overview
src/ask_my_docs/cli.py	Main runtime entrypoints	ingest, ask, metrics_summary, eval, _build_pipeline	DEFAULT_EVAL_OUTPUT, DEFAULT_BASELINE_PATH
src/ask_my_docs/settings.py	Typing settings and defaults from env	PricingConfig, RetrievalConfig, AppSettings, get_settings	env_prefix="ASK_MY_DOCS_", paths/timeouts/retrieval/pricing defaults
src/ask_my_docs/shared.py	Shared data contracts across modules	Document, Chunk, RetrievedChunk, RAGAnswer	timestamp_utc, default ISO timestamp
src/ask_my_docs/pipeline.py	Pipeline only request orchestration	RAGPipeline.answer, update_quality_metrics, _extract_citation	CITATION_RE, stores_raw_questions behavior

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
src/ask_my_docs/retrieval/hybrid.py	Document loading, chunking, embedding, hybrid search, artifact I/O	load_documents, HybridRetriever, HashingEmbedder	RetrievalConfig: from_document_path, load/save/search, embedding dim, top-k
src/ask_my_docs/llm/ollama.py	Local Ollama model resolution and generation API integration	parse_ollama_listbase_url, resolve_ollama_model, OllamaGenerator.generate	base_url, timeout_seconds, generate_timeout_seconds, configured_model
src/ask_my_docs/observability/metrics.py	Resiliability: summarize request metrics in DuckDB	RequestMetricRecorder, MetricsStore.record	update_quality/summarize p50/p95 quantiles, upsert by request_id
src/ask_my_docs/observability/tracing.py	OpenTelemetry: setup + JSONL span export	InspireSpanExporter, configure_tracing	Trace exporter, one-time global tracer config
src/ask_my_docs/observability/cost_tokens.py	Usability: cost-based cost estimation	TokenPricing, CostCalculator.estimate	prompt_cost_per_1k_tokens, completion_cost_per_1k_tokens
src/ask_my_docs/evaluation/runner.py	Evaluation: dataset I/O, per-example execution, aggregate report	EvalExample, EvalReport, load_eval_examples, run_evaluation, save_eval_report	Aggregate keys: F1/EM/recall/latency/cost/tokens
src/ask_my_docs/evaluation/metrics.py	Evaluation: quality/retrieval metric calculations	exact_match_score, token_f1_score, retrieval_recall_norm	Tokenization-based normalization via tokenize
src/ask_my_docs/evaluation/gating.py	Evaluation: baseline-relative regression checks	GateConfig, load_gate_config, evaluate_gate, GateResult	YAML thresholds in configs/regression_thresholds.yaml
configs/regression_thresholds.yaml	Config: thresholds	N/A (data file)	absolute + regression thresholds
data/eval/eval_set.jsonl	Evaluation: dataset	N/A (data file)	Per-line keys: question, reference_answer, expected_doc_ids
data/docs/	Source knowledge docs for indexing	N/A (content only)	.md/.txt files become doc_id + chunks

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
<code>.github/workflows/CI.yml</code>	Enforcement of quality/type/tests/gate	GitHub Actions steps for lint, mypy, tests, ingest, eval, gate	Uses <code>uv</code> , baseline file, thresholds file
<code>tests/</code>	Behavioral contracts and edge cases	<code>test_pipeline.py</code> , <code>test_retrieval.py</code> , etc.	Encodes expected error handling and metric behavior
<code>notebooks/tutorial/observability_on_nlp.ipynb</code>	Introduces tutorial path mirroring CLI/runtime	<code>NumberPy</code> calls retrieval, pipeline, eval, gating APIs	Uses <code>ASK_MY_DOCS_*</code> env overrides for notebook artifacts

Module 3: Core Execution Flows

Flow A: Build retrieval index (ingest)

Entry point:

- CLI command: `ask-my-docs ingest --docs-dir data/docs`
- Function: `ingest(...)` in `src/ask_my_docs/cli.py`

Step-by-step:

1. Resolve source directory:
 - `source_dir = docs_dir` or `settings.docs_dir`
2. Load documents:
 - `documents = load_documents(source_dir)`
 - `load_documents` recursively reads `.md` and `.txt` files.
 - `doc_id` is derived from relative path stem and sanitized by `_sanitize_doc_id`.
 - Duplicate `doc_id` collisions are disambiguated by a short BLAKE2b suffix.
3. Build hybrid retriever:
 - `HybridRetriever.from_documents(documents=documents, config=settings.retrieval)`
 - Internal stages:
 - `_chunk_document` using `chunk_size_tokens` and `chunk_overlap_tokens`.
 - `HashingEmbedder.embed` for deterministic dense vectors.
 - BM25 index via `BM25Okapi` and semantic index via `faiss.IndexFlatIP`.
4. Persist index artifacts:
 - `retriever.save(settings.index_dir)` writes:
 - `chunks.jsonl`

- `embeddings.npy`
- `semantic.index`
- `manifest.json`

Short code fragment:

```
documents = load_documents(source_dir)
retriever = HybridRetriever.from_documents(documents=documents, config=settings.retrieval)
retriever.save(settings.index_dir)
```

Input/Output shape details:

- Input docs: plain Markdown/TXT files.
- Document object shape:
- `doc_id`: `str`
- `text`: `str`
- `metadata`: `dict[str, str]` (includes `{"path": "relative/path.md"}`)
- `chunks.jsonl` line shape:
- `chunk_id`: `str`
- `doc_id`: `str`
- `text`: `str`
- `metadata`: `dict[str, str]` (includes `start_word`, `path`)
- `manifest.json` keys:
- `chunk_count`, `embedding_dim`, `lexical_weight`, `semantic_weight`

Flow B: Single Q&A request (ask)

Entry point:

- CLI command: `ask-my-docs ask "... --top-k 4 --output artifacts/sample_answer.json`
- Function: `ask(...)` in `src/ask_my_docs/cli.py`

Step-by-step:

1. Build runtime pipeline:
 - `_build_pipeline()` creates:
 - tracer: `configure_tracing(...)`
 - metrics store: `MetricsStore(db_path=...)`
 - retriever: `HybridRetriever.load(...)`
 - cost calculator: `CostCalculator(TokenPricing(...))`
 - generator: `OllamaGenerator(...)`
 - pipeline: `RAGPipeline(...)`
2. Execute request:
 - `result = pipeline.answer(question=question, top_k=...)`
 - `RAGPipeline.answer` does:
 - start trace span `rag.request`
 - retrieve chunks: `self._retriever.search(...)`

- generate answer: `self._generator.generate(...)`
 - compute cost: `self._cost_calculator.estimate(...)`
 - extract citations: `_extract_citations(...)`
 - write one `RequestMetricRecord` row
3. Optional JSON export:
- `payload = pipeline.to_payload(result)`
 - `write_json(output, payload)` if `--output` is provided.
4. Console summary:
- prints answer text, model, citations, latency, estimated cost.

Short code fragment:

```
result = pipeline.answer(question=question, top_k=top_k)
payload = pipeline.to_payload(result)
write_json(output, payload)
```

Input/Output shape details:

- Input: `question: str, top_k: int`.
- Returned object: `RAGAnswer` fields:
 - `request_id, trace_id, model_name, question, answer, citations`
 - `retrieved_doc_ids, prompt_tokens, completion_tokens, total_tokens`
 - `estimated_cost_usd, latency_ms, retrieval_latency_ms, generation_latency_ms`
 - `retrieval_recall_at_k` (optional), `timestamp_utc`

Example serialized payload keys (from `artifacts/notebook_tutorial/sample_answer.json`):

```
{
  "answer": "...",
  "citations": ["billing_disputes"],
  "completion_tokens": 350,
  "estimated_cost_usd": 0.0002682,
  "generation_latency_ms": 29088.41,
  "latency_ms": 29088.981,
  "model_name": "gemma4:12b",
  "prompt_tokens": 388,
  "question": "What is the SLA for invoice disputes?",
  "request_id": "...",
  "retrieval_latency_ms": 0.436,
  "retrieval_recall_at_k": null,
  "retrieved_doc_ids": ["billing_disputes", "onboarding_handover", "incident_sla", "access_c"],
  "timestamp_utc": "2026-06-12T03:10:38+00:00",
  "total_tokens": 738,
  "trace_id": "..."
}
```

Flow C: Observability summary (metrics-summary)

Entry point:

- CLI command: `ask-my-docs metrics-summary --limit 200`
- Function: `metrics_summary(...)` in `src/ask_my_docs/cli.py`

Step-by-step:

1. Open `MetricsStore` with configured DuckDB path.
2. Call `summary = metrics_store.summarize(limit=limit)`.
3. `summarize` computes aggregates and quantiles (`quantile_cont` for p50 and p95).
4. Print key-value lines.

Output fields printed:

- `request_count`
- `latency_p50_ms`
- `latency_p95_ms`
- `avg_cost_usd`
- `avg_retrieval_recall_at_k`
- `avg_answer_f1`
- `avg_exact_match`

DuckDB row schema (`rag_request_metrics`) includes:

- request metadata: `request_id`, `trace_id`, `model_name`, `question`, `timestamp_utc`
- latency/cost/tokens: `latency_ms`, `retrieval_latency_ms`, `generation_latency_ms`, `prompt_tokens`, `completion_tokens`, `total_tokens`, `estimated_cost_usd`
- quality fields: `retrieval_recall_at_k`, `answer_f1`, `exact_match`

Flow D: Evaluation and regression gate (eval)

Entry point:

- CLI command: `ask-my-docs eval --eval-path data/eval/eval_set.jsonl --output ... [--gate ...]`
- Function: `eval(...)` in `src/ask_my_docs/cli.py`

Step-by-step:

1. Load eval set:
 - `examples = load_eval_examples(dataset_path)`
 - JSONL line keys must be:
 - `question: str`
 - `reference_answer: str`
 - `expected_doc_ids: list[str]`
2. Run batch evaluation:

- `report = run_evaluation(pipeline=pipeline, examples=examples, top_k=settings.retrieval.top_k)`
 - For each sample:
 - call `pipeline.answer(...)`
 - compute `answer_f1` and `exact_match`
 - call `pipeline.update_quality_metrics(...)`
 - append per-example row
3. Save report:
- `save_eval_report(path=output, report=report)`
4. Optional baseline update:
- if `--set-baseline`, copy output report to baseline path.
5. Optional gate:
- load config: `load_gate_config(thresholds_path)`
 - parse baseline aggregate metrics if file exists
 - evaluate: `evaluate_gate(current_metrics, baseline_metrics, config)`
 - fail with exit code 1 if gate fails.

Short code fragment:

```
gate_result = evaluate_gate(
    current_metrics={key: float(value) for key, value in report.aggregate.items()},
    baseline_metrics=baseline_metrics,
    config=gate_config,
)
```

Evaluation report shape:

- Top-level:
- `aggregate`: `dict[str, float]`
- `per_example`: `list[dict[str, object]]`
- `aggregate` keys:
- `num_examples`, `answer_f1_mean`, `exact_match_mean`, `retrieval_recall_at_k_mean`
- `latency_p50_ms`, `latency_p95_ms`, `avg_cost_usd`, `avg_tokens`
- `per_example` row keys:
- `question`, `reference_answer`, `predicted_answer`, `model_name`
- `citations`, `retrieved_doc_ids`
- `retrieval_recall_at_k`, `answer_f1`, `exact_match`
- `latency_ms`, `retrieval_latency_ms`, `generation_latency_ms`
- `estimated_cost_usd`, `total_tokens`

Regression threshold file shape (`configs/regression_thresholds.yaml`):

- `absolute`:
- `min_answer_f1`
- `min_retrieval_recall_at_k`

- `max_latency_p95_ms`
- `max_avg_cost_usd`
- `regression:`
- `max_answer_f1_drop`
- `max_retrieval_recall_drop`
- `max_latency_p95_increase_ms`
- `max_avg_cost_increase_usd`

Flow E: Trace export internals

`configure_tracing(service_name, trace_path)` configures a global OpenTelemetry tracer provider and registers `BatchSpanProcessor(JsonLineSpanExporter(trace_path))`.

Each exported span JSON line has keys:

- `name, trace_id, span_id, parent_span_id`
- `start_time_unix_ns, end_time_unix_ns, duration_ms`
- `status, attributes`

This enables offline debugging of span-level behavior without external tracing infrastructure.

Module 4: Setup & Run Guide

4.1 Prerequisites

From repository `manifests/docs`:

- OS: Linux/macOS (as documented in `README.md`)
- Python: `>=3.11` (from `pyproject.toml`)
- Package manager/environment: `uv`
- Ollama installed and running locally
- At least one local Ollama model available in `ollama list`

4.2 Install on a clean machine

```
git clone https://github.com/pypi-ahmad/production-rag-ask-my-docs.git
cd production-rag-ask-my-docs
```

```
uv venv --python 3.12.10
source .venv/bin/activate
uv sync --all-extras
```

4.3 Configuration

Settings source:

- `src/ask_my_docs/settings.py` (`AppSettings`)
- env prefix: `ASK_MY_DOCS_`
- `.env` is supported (`env_file=".env"`)

Key environment variables used by this project:

- ASK_MY_DOCS_SERVICE_NAME (default ask-my-docs-rag)
- ASK_MY_DOCS_OLLAMA_BASE_URL (default `http://127.0.0.1:11434`)
- ASK_MY_DOCS_OLLAMA_MODEL (optional explicit model)
- ASK_MY_DOCS_OLLAMA_TIMEOUT_SECONDS (default 120.0)
- ASK_MY_DOCS_OLLAMA_LIST_TIMEOUT_SECONDS (default 10.0)
- ASK_MY_DOCS_DOCS_DIR (default data/docs)
- ASK_MY_DOCS_EVAL_PATH (default data/eval/eval_set.jsonl)
- ASK_MY_DOCS_ARTIFACTS_DIR (default artifacts)
- ASK_MY_DOCS_INDEX_DIR (default artifacts/index)
- ASK_MY_DOCS_TRACES_PATH (default artifacts/observability/traces.jsonl)
- ASK_MY_DOCS_METRICS_DB_PATH (default artifacts/observability/metrics.duckdb)
- ASK_MY_DOCS_THRESHOLDS_PATH (default configs/regression_thresholds.yaml)
- ASK_MY_DOCS_OBSERVABILITY_STORE_RAW_QUESTIONS (default false)

Minimal example:

```
export ASK_MY_DOCS_OLLAMA_BASE_URL=http://127.0.0.1:11434
export ASK_MY_DOCS_OLLAMA_MODEL=gemma4:12b
```

4.4 Typical command sequence (main runtime path)

1. Build index:

```
uv run ask-my-docs ingest --docs-dir data/docs
```

2. Ask a question:

```
uv run ask-my-docs ask "What is the SLA for invoice disputes?" --top-k 4
```

3. Ask and save JSON payload:

```
uv run ask-my-docs ask "What is the SLA for invoice disputes?" \
--top-k 4 \
--output artifacts/sample_answer.json
```

4. Summarize metrics:

```
uv run ask-my-docs metrics-summary
```

5. Run evaluation:

```
uv run ask-my-docs eval \
--eval-path data/eval/eval_set.jsonl \
--output artifacts/eval/current_metrics.json
```

6. Set baseline:

```
uv run ask-my-docs eval \
--eval-path data/eval/eval_set.jsonl \
--output artifacts/eval/current_metrics.json \
```

```
--set-baseline \  
--baseline artifacts/baseline_metrics.json
```

7. Apply regression gate:

```
uv run ask-my-docs eval \  
--eval-path data/eval/eval_set.jsonl \  
--output artifacts/eval/gate_metrics.json \  
--baseline artifacts/baseline_metrics.json \  
--thresholds configs/regression_thresholds.yaml \  
--gate
```

4.5 Migration/seeding notes

- Retrieval “seeding” is the `ingest` step that creates index artifacts in `ASK_MY_DOCS_INDEX_DIR`.
- There is no separate migration command for DuckDB.
- `MetricsStore._init_schema()` creates `rag_request_metrics` automatically.
- It also performs an internal migration path (`_migrate_schema_to_v2`) when an old table lacks primary key constraints.

4.6 CI execution path

From `.github/workflows/ci.yml`:

1. `uv sync --all-extras`
2. `uv run ruff check src tests`
3. `uv run mypy src`
4. `uv run pytest -q`
5. `uv run ask-my-docs ingest --docs-dir data/docs`
6. `uv run ask-my-docs eval ... --output artifacts/eval/ci_metrics.json`
7. `uv run ask-my-docs eval ... --gate`

This is the repository’s canonical quality gate pipeline.

Module 5: Study Plan & Practice Exercises

5.1 Ordered study plan for a new learner

Recommended read order:

1. `README.md` and `pyproject.toml`
 - Understand project goals, dependencies, and command surface.
2. `src/ask_my_docs/settings.py` and `src/ask_my_docs/models.py`
 - Learn config defaults and shared data contracts.
3. `src/ask_my_docs/retrieval/hybrid.py`

- Understand indexing/search mechanics and artifact formats.
- 4. `src/ask_my_docs/llm/ollama.py`
 - Understand model discovery, prompt construction, and generation API calls.
- 5. `src/ask_my_docs/pipeline.py`
 - Learn how retrieval, generation, cost, traces, and metrics are stitched together.
- 6. `src/ask_my_docs/observability/*.py`
 - Study persistence schema, quantile summary, and trace export design.
- 7. `src/ask_my_docs/evaluation/*.py` and `configs/regression_thresholds.yaml`
 - Understand metrics computation, reporting, and gate decisions.
- 8. `src/ask_my_docs/cli.py`
 - Map everything into production-facing commands.
- 9. `tests/`
 - Validate your understanding against explicit behavioral assertions.
- 10. `notebooks/tutorial_observability_ollama.ipynb`
 - See the same concepts in an interactive walkthrough.

5.2 Practice exercises

1. Trace one `ask` request end-to-end. Task: Starting at `cli.ask`, list every major function called until metrics are written.
2. Explain how citation extraction works. Task: In `RAGPipeline._extract_citations`, describe when regex matches are used and when fallback citations are used.
3. Analyze retrieval scoring. Task: In `HybridRetriever.search`, explain how lexical and semantic scores are normalized and combined.
4. Explain document identity guarantees. Task: In `load_documents`, explain how duplicate `doc_id` collisions are prevented.
5. Decode evaluation dataset contract. Task: Write the exact required keys and types for one line of `data/eval/eval_set.jsonl`.
6. Decode gate decision logic. Task: In `evaluate_gate`, identify all absolute checks and all regression checks.
7. Explain privacy behavior for stored questions. Task: In `RAGPipeline._question_for_storage`, explain the default redaction format and when raw question text is stored.

8. Explain metrics DB upsert semantics. Task: In `MetricsStore.record`, describe what happens when a `request_id` already exists.
9. Explain model selection precedence. Task: In `resolve_ollama_model`, describe priority order when `configured_model` is set vs unset.
10. Explain CI gate workflow. Task: Using `.github/workflows/ci.yml`, list the sequence from static checks to gate pass/fail.

5.3 Solution outlines (brief)

1. End-to-end ask flow
 - `cli.ask` -> `_build_pipeline` -> `RAGPipeline.answer` -> `HybridRetriever.search` -> `OllamaGenerator.generate` -> `CostCalculator.estimate` -> `MetricsStore.record` -> optional `write_json`.
2. Citation extraction
 - Regex `_CITATION_RE` extracts bracketed ids like `[doc_id: billing_disputes]`; only ids present in `retrieved_doc_ids` are accepted. If none match, fallback returns first unique two retrieved doc IDs.
3. Retrieval scoring
 - BM25 and FAISS scores are independently normalized with `_normalize`, then combined via `lexical_weight * lexical_norm + semantic_weight * semantic_norm` and ranked descending.
4. Doc ID uniqueness
 - Path-derived stem is sanitized; if already used, code appends a short BLAKE2b suffix from relative path; if collision still happens, it raises `ValueError`.
5. Eval JSONL schema
 - `{"question": str, "reference_answer": str, "expected_doc_ids": list[str]}` per line.
6. Gate checks
 - Absolute: min F1, min recall@k, max p95 latency, max avg cost.
 - Regression: max drop in F1/recall, max increase in p95 latency/avg cost vs baseline.
7. Question redaction
 - Default stores `<redacted:sha256:<16-hex>:len=<n>>`; raw text only when `store_raw_questions=True` (`ASK_MY_DOCS_OBSERVABILITY_STORE_RAW_QUESTIONS=true`).
8. Metrics upsert

- Insert uses `ON CONFLICT(request_id) DO UPDATE`; existing row is replaced with latest values for all tracked fields.
9. Model selection
- If `configured_model` is set, it must exist in `ollama list` output or runtime error is raised.
 - If unset, first local model (`size != "-"`) is preferred; otherwise first listed model is used.
10. CI workflow
- Checkout -> setup uv -> dependency sync -> lint -> type check -> unit tests -> ingest -> eval candidate metrics -> eval with `--gate`.

Understanding Checklist

Use this checklist to verify mastery:

- Can you explain `ingest` from file read to saved index artifacts (`chunks.jsonl`, `embeddings.npy`, `semantic.index`, `manifest.json`)?
- Can you explain one `ask` request end-to-end, including retrieval, generation, citations, cost estimation, tracing, and metric persistence?
- Can you describe the exact `RAGAnswer` payload keys and where each value is produced?
- Can you explain how quality metrics (`F1`, `exact_match`, `recall@k`) are computed and where they are stored?
- Can you explain the difference between absolute and regression gate checks in `evaluate_gate`?
- Can you list the environment variables needed to run the project on a clean machine?
- Can you explain how DuckDB schema initialization/migration is handled automatically?
- Can you trace where CI enforces quality and where it can fail the pipeline?

If you can answer all eight without opening files, you are ready to contribute changes confidently.